

CS-4032: Web Programming

Assignment #02 — 2D Cricket Web Application

Name: Fatima Ishtiaq
Roll Number: 23i-0696
Section: C

1. Game Logic and Working Mechanism

1.1 Overview

Cricket Bash is a single-player 2D batting simulation game built using React, HTML5 Canvas, and JavaScript. The player acts as a batsman and plays a fixed innings of 2 overs (12 balls) with a maximum of 2 wickets. The game ends when all 12 balls are bowled or both wickets are lost.

1.2 Game Flow

The game follows a strict sequential flow for each ball:

- Player selects Batting Style: Aggressive or Defensive (only selectable before a ball starts).
- Player clicks BOWL — a bowling animation plays where the ball travels from the bowler (top of pitch) to the batsman (bottom of pitch) using `requestAnimationFrame`.
- Once the ball reaches the batsman, the Power Bar slider activates and begins moving continuously from left to right across the bar.
- Player clicks PLAY SHOT — the slider freezes at its current position. The position value (0.0 to 1.0) is used to determine the outcome.
- A bat swing animation plays, then the result (runs scored or wicket) is evaluated and displayed.
- The scoreboard updates: runs, wickets, balls bowled, overs, and strike rate.
- Dynamic commentary is shown for every outcome.
- This cycle repeats until the innings ends (12 balls bowled or 2 wickets fallen).
- A Game Over screen shows final statistics with a Play Again button.

1.3 Outcome Determination Logic

The outcome is NOT random. It is determined entirely by the position of the slider when the player clicks PLAY SHOT. The slider position is a value between 0.0 and 1.0. Each outcome occupies a proportional segment of this range based on its probability. The function `getOutcomeFromSlider()` iterates through each outcome's cumulative probability boundary and returns the outcome whose segment contains the slider position.

Example: For Aggressive style, if the slider stops at 0.55:

- Wicket occupies 0.00 – 0.30
- 0 Runs occupies 0.30 – 0.40
- 1 Run occupies 0.40 – 0.50
- 2 Runs occupies 0.50 – 0.60 ← slider at 0.55 falls here

Result: 2 Runs are scored.

1.4 Game Rules Summary

Rule	Value
Total Overs	2 (12 balls)

Total Wickets	2
Batting Styles	Aggressive, Defensive
Outcome Method	Slider Position (Not Random)
Commentary	Dynamic, per-outcome (5 lines each)

2. Mapping of Probabilities to the Power Bar

2.1 Design Principle

The Power Bar is a horizontal strip divided into colored segments. Each segment represents one possible outcome. The width of each segment is directly proportional to its probability. For example, an outcome with probability 0.30 occupies exactly 30% of the bar's total width. This ensures visual accuracy — the player can estimate their chances by looking at the bar.

The slider moves at a fixed speed continuously from left to right (wrapping at 1.0 back to 0.0). Clicking PLAY SHOT freezes the slider. The frozen position is checked against each segment's cumulative probability boundary to find the outcome.

2.2 Aggressive Batting — Probability Distribution

Aggressive style prioritizes high reward at high risk. The wicket region is large (30%), but so are boundaries (4 Runs: 20%, 6 Runs: 15%).

Outcome	Probability	Bar %	Range (0–1)
Wicket (W)	0.30	30%	0.00 – 0.30
0 Runs	0.10	10%	0.30 – 0.40
1 Run	0.10	10%	0.40 – 0.50
2 Runs	0.10	10%	0.50 – 0.60
3 Runs	0.05	5%	0.60 – 0.65
4 Runs	0.20	20%	0.65 – 0.85
6 Runs	0.15	15%	0.85 – 1.00
TOTAL	1.00	100%	—

2.3 Defensive Batting — Probability Distribution

Defensive style minimizes risk. The wicket region is small (15%), and most outcomes cluster around safe 0–2 run zones. Boundaries (4, 6) are rare.

Outcome	Probability	Bar %	Range (0–1)
Wicket (W)	0.15	15%	0.00 – 0.15
0 Runs	0.20	20%	0.15 – 0.35
1 Run	0.30	30%	0.35 – 0.65
2 Runs	0.20	20%	0.65 – 0.85
3 Runs	0.05	5%	0.85 – 0.90

4 Runs	0.07	7%	0.90 – 0.97
6 Runs	0.03	3%	0.97 – 1.00
TOTAL	1.00	100%	—

2.4 Power Bar Visual Design

Each segment in the power bar is rendered as a CSS flex div with width set to $\text{seg.prob} * 100 + \%$. The segments are colored distinctly for readability:

- Wicket — Red gradient (danger indicator)
- 0 Runs — Dark grey (dead ball)
- 1 Run — Steel blue
- 2 Runs — Green
- 3 Runs — Purple
- 4 Runs — Orange
- 6 Runs — Gold

A white slider needle (#pb-slider) is positioned using $\text{left: sliderPos} * 100 + \%$ and animates via a `requestAnimationFrame` loop incrementing `sliderRef.current` by 0.008 per frame. The slider wraps from 1.0 back to 0.0 continuously until the player clicks.

3. Implementation of Animations

3.1 Rendering Architecture

All game visuals are rendered on an HTML5 Canvas element using a persistent render loop (`requestAnimationFrame`). The canvas is redrawn every frame using the `drawField()` function, which reads live refs (`bowlAnimRef`, `batSwingRef`, `ballPosRef`) to determine current positions. This decouples animation state from React's render cycle.

3.2 Bowling Animation

When the player clicks BOWL, the `startBowling()` function launches an animation loop that increments `bowlAnimRef.current` from 0 to 1 at a rate of 0.025 per frame (approximately 40 frames = ~0.67s at 60fps). The ball's position is interpolated linearly:

```
ballX = startX + (endX - startX) * t
ballY = startY + (endY - startY) * t
```

The ball travels from the bowler's position (top of pitch, ~42% vertical) down to the batsman's crease (~74% vertical). When `t` reaches 1.0, the phase switches to 'ready' and the power bar slider activates.

3.3 Bat Swing Animation

When the player clicks PLAY SHOT, a bat swing animation is triggered. The variable `batSwingRef.current` increments from 0 to 1 at 0.06 per frame (~17 frames). In the `drawBatsman()` canvas function, the bat arm is rotated using:

```
ctx.rotate(-swing + 0.3)
```

where `swing = batSwingRef.current * PI * 0.75`, producing a sweeping 135-degree arc. The batsman's helmet color also changes (green for defensive, blue for aggressive) to visually confirm style selection. After the swing completes, outcome processing triggers a 400ms delay before the result is shown and the scoreboard updates.

3.4 Slider Animation

The power bar slider runs in a completely independent `useEffect` animation loop (separate from the main render loop). It increments `sliderRef.current` by 0.008 per frame continuously, wrapping at 1.0. This ref-based approach avoids React re-renders on every frame. The visual position is updated via a single `setSliderPos` call per frame, which only re-renders the slim slider element.

3.5 Animation Synchronization

All animations are synchronized through a `phaseRef` state machine with the following states:

- `idle` — No animation running. Player can select style and click BOWL.
- `bowling` — Ball travel animation in progress. Controls locked.
- `ready` — Ball delivered. Slider active. Player must click PLAY SHOT.
- `animating` — Bat swing in progress. Controls locked.
- `gameover` — Innings ended. Game Over screen shown.

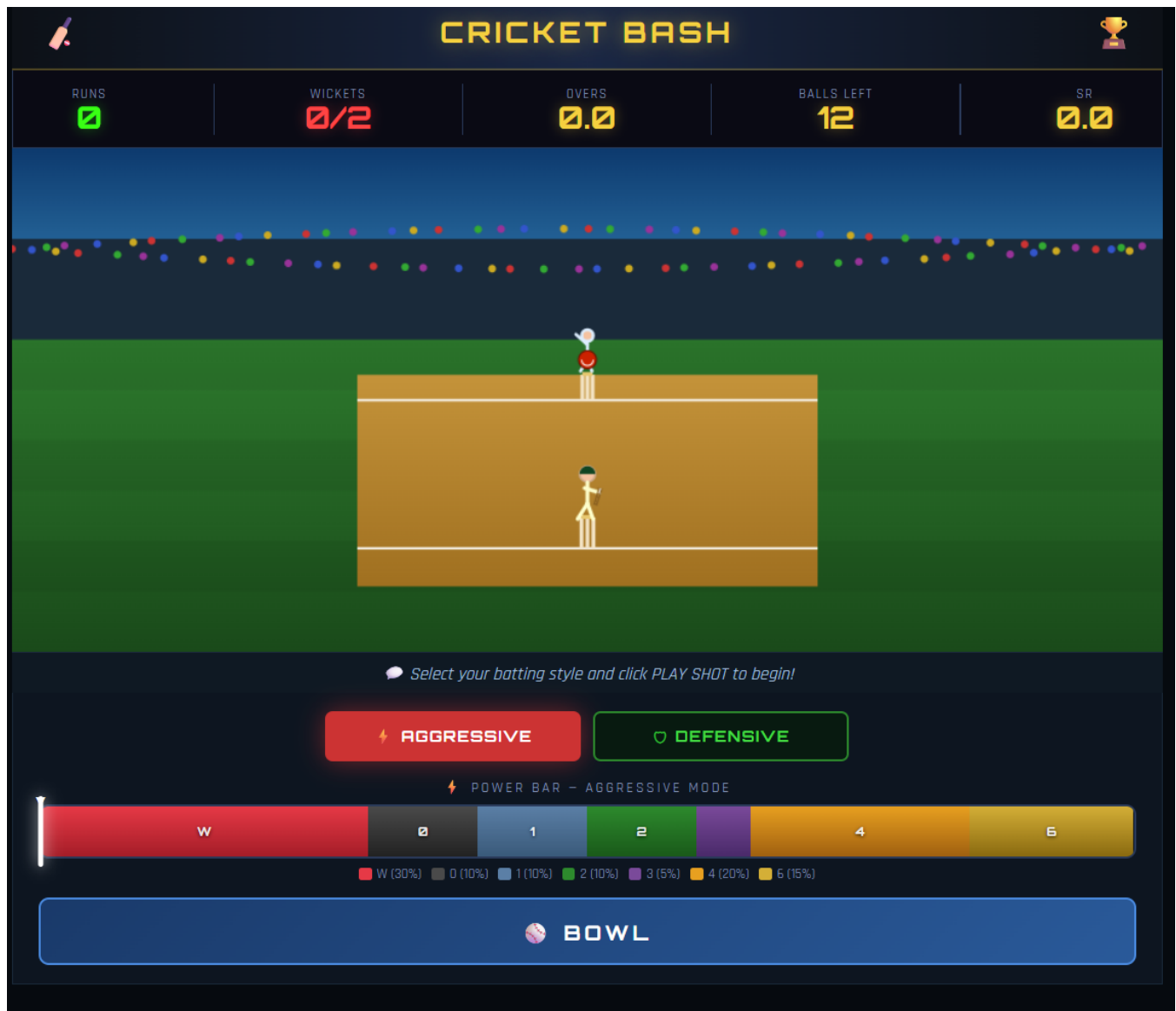
State transitions are managed in `phaseRef` (a mutable ref to avoid stale closures) and mirrored in `gamePhase` React state for UI rendering (button labels, disabled states).

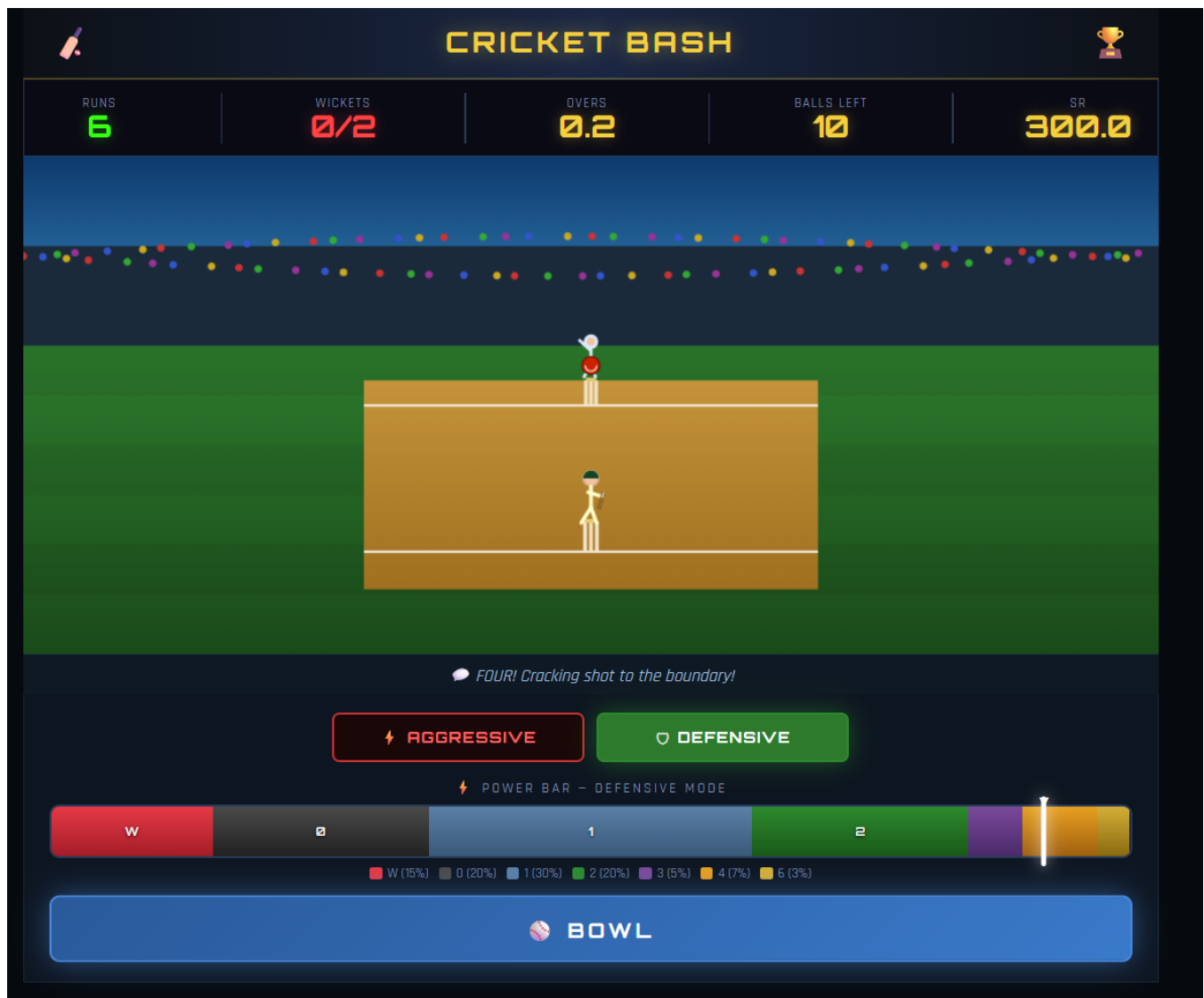
3.6 2D Field Drawing

The `drawField()` function renders the following layers on every frame:

- Sky gradient (dark blue to light blue) — top 38% of canvas
- Crowd stands (dark panel with 80 randomized colored circles for crowd heads)
- Outfield grass (dark-to-light green gradient with 6 alternating stripes)
- Pitch (brown rectangle, 30–70% horizontal, 45–87% vertical)
- Crease lines (white lines at bowling and batting ends)
- Stumps at both ends (3 vertical rects + bail bar)
- Bowler figure (stick figure, top of pitch)
- Cricket ball (red filled circle with white seam arc)
- Batsman (stick figure with helmet, colored arm + bat blade + handle)

4. Gameplay Screenshots





5. Technology Stack

Technology	Usage
React 18	UI state management (useState, useEffect, useRef, useCallback), component lifecycle
HTML5 Canvas	2D cricket field rendering — pitch, stumps, ball, batsman, bowler, crowd
CSS3	Scoreboard, power bar segments, slider, animations, responsive layout, Google Fonts
JavaScript ES6+	requestAnimationFrame game loop, probability logic, animation interpolation
Orbitron / Rajdhani	Google Fonts — Orbitron for scoreboards/headings, Rajdhani for body text

6. Bonus Features Implemented

6.1 Dynamic Commentary System

A COMMENTARY object stores 5 hardcoded commentary strings for each outcome (W, 0, 1, 2, 3, 4, 6). After every ball, getRandCommentary(key) selects one at random using $\text{Math.floor}(\text{Math.random()} * \text{arr.length})$ and displays it in the commentary bar below the canvas. Sample lines:

- Wicket: "BOWLED! What a delivery! The stumps are shattered!"
- Six: "SIX! MASSIVE HIT! Into the stands!"
- Dot: "Dot ball. Good tight bowling."

6.2 Strike Rate Display

The scoreboard includes a live Strike Rate (SR) calculated as $(\text{runs} / \text{ballsBowled}) * 100$, updating after every ball.

6.3 Smooth Synchronized Animations

All animations use requestAnimationFrame for smooth 60fps playback. The bowling, batting, and slider animations are precisely synchronized via the phaseRef state machine to prevent any overlap or race conditions.

6.4 Responsive Design

The canvas and layout use percentage widths with CSS aspect-ratio, making the game fully responsive on mobile screens. A ResizeObserver updates canvas dimensions on window resize.